

Monitoreo de cadena por galgas — DREYFUS

Esquemáticos y funcionamiento · transportador REDLER RPRB3

Louis Dreyfus Company · parada de planta octubre 2026 · Matías Alegre — GIMAP / UTN FRBB · documento generado el 19-ago-2026

1. Qué hace el sistema

Detectar la **rotura de una cadena** en el transportador REDLER, cuyo eje gira a **20 RPM**. Se comparan las deformaciones que miden **dos galgas** (A y B), una por cadena: si una se corta, cambia el reparto de carga y las dos señales dejan de parecerse.

Dato medido en campo	Qué implica
Golpeteo de eslabones: 0,397 Hz (por FFT sobre datos reales)	Muestrear a 5-10 Hz sobra. La velocidad no es el problema.
Ruido 1,08 mV sobre eventos de 20 mV	Señal/ruido de 4,7×: se ve el golpe grande, no un corte chico. Este sí es el problema.
Sensibilidad de la galga: 1 µε = 1,50 µV	Se miden microvolts: manda el ruido y la deriva, no los bits.

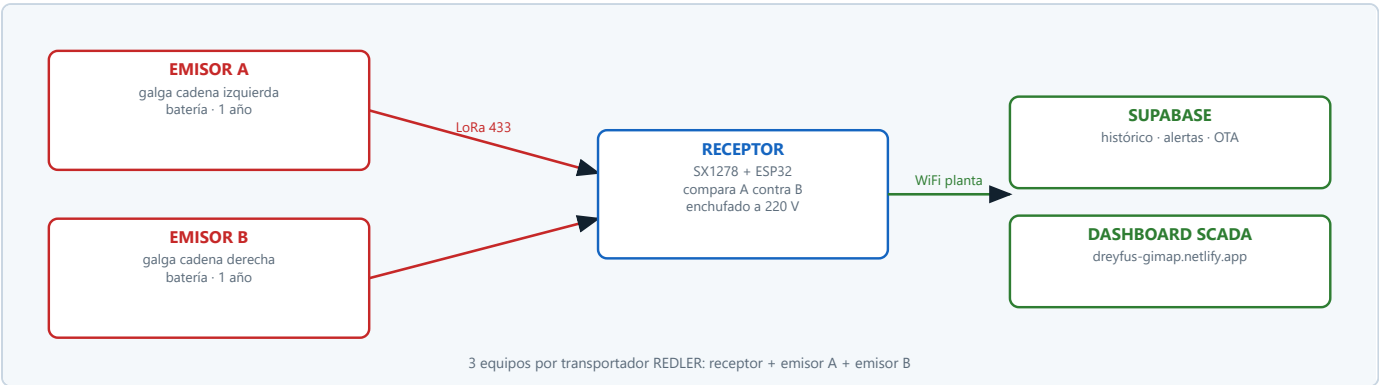
2. Atención: hay DOS diseños vivos

Antes de leer los esquemáticos. El proyecto tiene una generación **construida y probada** (v2) y otra **diseñada y auditada pero sin construir** (v3). No son alternativas teóricas: son dos estados distintos de madurez, y conviene no confundirlos al mostrar el sistema.

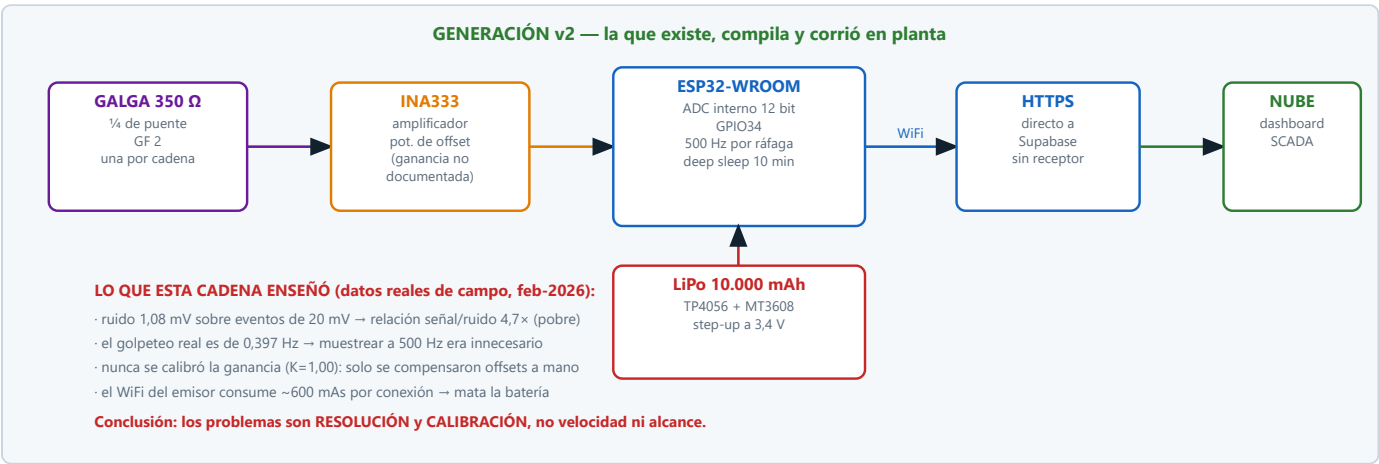
	v2 — lo que EXISTE	v3 — lo que se DISEÑÓ
Estado	Código real, compila, corrió en planta (feb-2026)	Solo papel. Cero líneas de código escritas.
Microcontrolador	ESP32-WROOM (con WiFi)	ATmega328P a 8 MHz
Medición	INA333 + ADC interno de 12 bits	AD7124-8 de 24 bits, ratiométrico
Radio	WiFi → HTTPS directo a la nube	LoRa 433 MHz → receptor → nube
Batería	LiPo 10.000 mAh	LiSOCl2 AA + supercapacitor
Autonomía	semanas	~1,3 años (proyectado)

Decisión vigente (08-jul-2026): se va con la **v3** a la parada de octubre, con la **v2 como plan B vivo** y un **checkpoint duro a mediados de septiembre**: si para esa fecha el nodo v3 no pasó banco (ruido, consumo en reposo medido, shunt-cal con A=B), va la v2 a la parada y la v3 queda como Proyecto Final.

3. Topología (v3)



4. Esquemático de la v2 — la cadena que existe hoy



Parámetros reales del firmware v2

Parámetro	Valor	Dónde está
Entrada de la galga	GPIO 34 (ADC1)	config.h
Frecuencia de muestreo	500 Hz	protocol.h
Ráfaga normal / en alerta	500 muestras (1 s) / 1000 (2 s)	protocol.h
Cada cuánto despierta	600 s normal · 60 vigilado · 10 en alerta	protocol.h
Umbral de auto-alarma	pico a pico > 40 mV	config.h
Qué se transmite	media, RMS, mín, máx, pico-pico, desvío (no la onda cruda)	supabase_client.cpp
Corte por batería baja	3,20 V	config.h

Detalle de implementación que vale recordar: el timer del ESP32 no baja de ~1220 Hz, así que 500 Hz no se pueden pedir directo. Se configura el timer a 1 MHz y se dispara la interrupción cada 2000 μ s. La rutina de interrupción hace una sola cosa: guardar la muestra en el buffer.

5. Cómo funciona la v2 — pseudocódigo

```
# El ESP32 no tiene bucle principal: hace todo al despertar y se vuelve a dormir.
```

al despertar del deep sleep:

```
leer de la memoria NVS: K, offset, umbrales, perfil de consumo  
conectar al WiFi          # ~600 mAs: el gasto más grande del ciclo
```

```
# — ráfaga de medición —
```

```
arrancar el timer a 500 Hz
```

```
tomar 500 muestras del ADC en la interrupción    # 1 segundo
```

```
# — descartar lo que no sirve —
```

```
para cada muestra:
```

```
    si está fuera de 0,30 a 3,20 V → descartarla    # el amplificador se saturó
```

```
contar cuántas se descartaron
```

```
# — resumir la ráfaga —
```

```
calcular media, RMS, mínimo, máximo, pico a pico y desvío
```

```
# — decidir si hay alarma —
```

```
si pico_a_pico > 40 mV:
```

```
    esperar 1,5 s y tomar OTRA ráfaga          # confirmación
```

```
    si la segunda también supera el umbral:
```

```
        marcar ALERTA y pasar a despertar cada 10 s
```

```
    # esta doble confirmación bajó los falsos positivos de 6 a 0
```

```
# — publicar —
```

```
POST a la nube con los estadísticos, la batería, el RSSI y el estado
```

```
si hay alerta nueva → POST del evento (con reintentos que sobreviven al sueño)
```

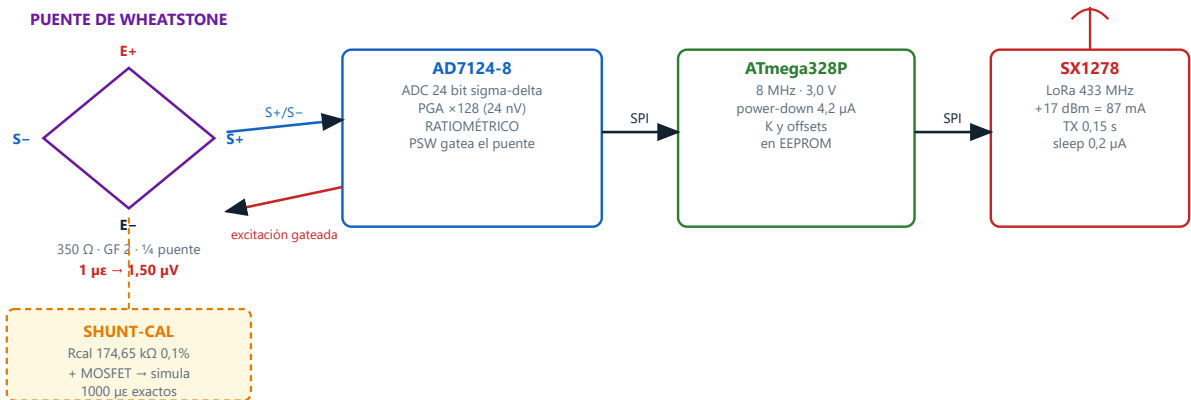
```
preguntar si hay comandos pendientes (cambio de umbral, actualización, etc.)
```

```
# — dormir —
```

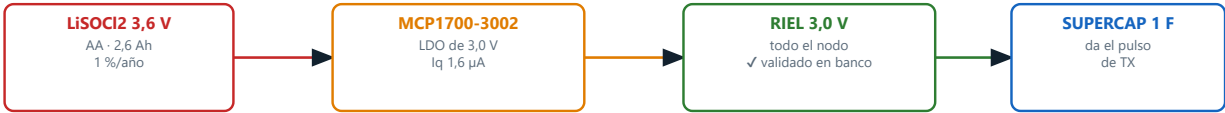
```
dormir 600 s    # o 60 si está vigilando, o 10 si está en alerta
```

6. Esquemático de la v3 — el diseño para octubre

GENERACIÓN v3 — el diseño para octubre (auditado, todavía sin construir)



CADENA DE ALIMENTACIÓN



LAS TRES DECISIONES QUE DEFINEN ESTE DISEÑO

1. ADC de 24 bits RATIOMÉTRICO: excita el puente con la misma tensión que usa de referencia → el error térmico se cancela solo.
2. PUENTE GATEADO: 350 Ω a 3,0 V son 8,6 mA permanentes. Se enciende 3 s cada 5 min y se apaga. Sin esto no hay batería que aguante.
3. LoRa EN VEZ DE WiFi en el emisor: despertar el WiFi cuesta ~600 mAs; una transmisión LoRa, ~20 mAs. Ese fue el asesino de la batería.

Regla del proyecto: ningún número entra al documento sin la página del datasheet al lado.

7. Cómo funcionaría la v3 — pseudocódigo del emisor

al encender por primera vez:

```
leer de la EEPROM: K del canal, offset, número de nodo
configurar el ADC: ganancia 128, modo de bajo consumo, referencia ratiométrica
# la pila de litio estuvo meses guardada y arranca "dura" (pasivada)
pedirle corriente unos segundos y verificar que se mantenga sobre 3,0 V
si no llega → no seguir: la pila está pasivada o agotada
```

cada 5 minutos:

```
despertar por el watchdog

encender el puente          # llave interna del ADC – acá se decide la batería
esperar a que se estabilice
tomar 15 muestras a 5 Hz    # 3 segundos
apagar el puente

deformación = (promedio - offset) × K
calcular también mínimo, máximo y desvío
leer la temperatura         # para compensar la deriva

si toca el autocontrol diario:
    activar el shunt-cal y comparar con el valor teórico
    marcar la bandera "calibración OK" o "revisar"

armar el paquete y transmitir por LoRa    # 87 mA durante 0,15 s
# el supercapacitor entrega ese pulso sin hundir el riel de 3,0 V
abrir una ventana corta de escucha        # por si el receptor manda una orden

dormir 5 minutos                        # en microamperes: acá vive el 99 % del tiempo
```

Pseudocódigo del receptor

cuando llega un paquete por LoRa:

```
verificar que sea de uno de nuestros nodos y guardarlo

si tengo lecturas recientes de A y de B:
    diferencia = |deformación_A - deformación_B|
    si la diferencia supera el umbral varias veces seguidas:
        ALARMA: el reparto de carga cambió → posible cadena cortada

si un nodo no reporta hace más de 20 minutos → avisar "nodo caído"
si su pila cae por debajo de 3,1 V          → avisar "cambiar pila"
si su bandera de calibración dice "revisar" → avisar

subir a la nube; si no hay internet, guardar y reintentar
```

8. Calibración shunt-cal

Permite verificar el sistema **en planta, sin instrumental**. Se conecta una resistencia de precisión en paralelo a un brazo del puente, lo que simula una deformación de valor conocido:

$$\epsilon_{\text{simulada}} = R_{\text{galga}} / (GF \times (R_{\text{galga}} + R_{\text{cal}}))$$
$$350 \, \Omega / (2 \times 175.000 \, \Omega) = 0,001 = 1000 \, \mu\epsilon \text{ exactos}$$

con $R_{\text{cal}} = 174,65 \, \text{k}\Omega$ al 0,1 %

Paso	Qué se hace
1	Cero: 30 s sin carga, se registra el offset.
2	Span: se activa el MOSFET del shunt y se mide el salto.
3	Se ajusta la K del canal para que el salto dé 1000 $\mu\epsilon$.
4	Verificación cruzada: A y B tienen que dar lo mismo.
5	K y offsets se guardan en la EEPROM.

Por qué es imprescindible: en la v2 la ganancia nunca se calibró (K quedó en 1,00) y solo se compensaron los offsets a mano, con un potenciómetro distinto en cada canal. Comparar A contra B sin las K reales **no es confiable** — y comparar A contra B es el método de detección.

9. Presupuesto de energía (v3)

Momento	Consumo	Duración
Durmiendo	4,2 μA el micro · 2 μA el ADC	99 % del tiempo
Midiendo (puente encendido)	8,6 mA — el consumidor dominante	3 s cada 5 min
Transmitiendo	87 mA a +17 dBm	0,15 s
Promedio	0,19 mA	
Autonomía	pila AA LiSOCI2 (2,6 Ah, útil ~2,1)	~1,3 años — el año se cumple con 30 % de margen

Nada de esto se da por bueno hasta medirlo. El consumo en reposo se verifica con un medidor de corriente antes de creer en las cuentas: si da miliamperes en vez de microamperes, algo quedó encendido. Es la lección del prototipo anterior, que se comió 8800 mAh en un día.

10. Estado real y qué falta

Tema	Estado
Ciclo completo v2 (medir → dormir → publicar → comandos → actualización remota)	Validado de punta a punta en banco
Dashboard SCADA conectado a la nube real	Funcionando
Cadena de alimentación de 3,0 V (v3)	Verificada en banco con radio transmitiendo
Receptor	Incompleto: hoy solo manda señal de vida
Prueba con galga física + amplificador	NUNCA se hizo: todo corrió con señal simulada
Prueba con batería real	NUNCA: el firmware finge 4,0 V en el banco
Front-end analógico de la v3 (shunt-cal, gateo, protecciones)	Sin diseñar — es la parte más delicada
Firmware de la v3	Sin empezar
Memoria del ESP32 en la v2	Al 94 % — riesgo para las actualizaciones remotas
Fecha exacta de la parada, quién instala	A definir con GIMAP y Dreyfus

11. Checklist del día de la instalación

#	Antes de dar por instalado un nodo
1	Despertar la pila de litio: pedirle corriente unos segundos y verificar que se mantenga sobre 3,0 V. Sin esto se apaga en el primer arranque.
2	Disparar el shunt-cal y confirmar que A y B dan lo mismo.
3	Verificar que el receptor recibe los dos nodos y con qué nivel de señal (el entorno metálico atenúa).
4	Confirmar que los datos llegan a la nube y que el tablero abre desde el celular.
5	Anotar hora de instalación y tensión inicial de cada pila .
6	Sellar el nodo y verificar que se lo puede despertar con el imán , sin abrirlo.

Fuentes: firmware y datos de campo del repositorio de galgas · estudio de ingeniería y auditoría adversarial contra hojas de datos oficiales. Los parámetros de la v2 salen del código; los de la v3, del diseño auditado.